

Practical 11

Task 1

Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```
GNU nano 7.2 Race_condit
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define NUM_THREADS 5
#define INCREMENTS 100000
int counter = 0; // Shared variable
void *increment_counter(void *arg)
{
    int i;
    for (i = 0; i < INCREMENTS; i++)
    {
        counter++;
    }
    return NULL;
}
int main()
{
    pthread_t threads[NUM_THREADS];
    int i;
    printf("Initial counter = %d\n", counter);
    /* Create threads */
    for (i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(
            &threads[i],
            NULL,
            increment_counter,
            NULL) != 0)
```

```
        {
            perror("pthread_create");
            return 1;
        }
    }
    /* Wait for all threads */
    for (i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }
    printf("Expected counter = %d\n",
        NUM_THREADS * INCREMENTS);
    printf("Actual counter = %d\n", counter);
    return 0;
}
```

Output:

```
root@Tejasri:~# nano race_condition.c
root@Tejasri:~# gcc race_condition.c -o race_condition
root@Tejasri:~# ./race_condition
Initial counter = 0
Expected counter = 500000
Actual counter   = 255250
root@Tejasri:~# |
```

Task 2

Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```
#include <stdio.h>
#include <pthread.h>
#define NUM_THREADS 5
#define INCREMENTS 100000
int counter = 0;
/* Create mutex */
pthread_mutex_t mutex;
void *increment_counter(void *arg)
{
    int i;
    for (i = 0; i < INCREMENTS; i++)
    {
        /* Lock before accessing shared variable */
        pthread_mutex_lock(&mutex);
        counter++;
        /* Unlock after updating */
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
int main()
{
    pthread_t threads[NUM_THREADS];
    int i;
    /* Initialize mutex */
    pthread_mutex_init(&mutex, NULL);
    printf("Initial counter = %d\n", counter);
    /* Create threads */
    for (i = 0; i < NUM_THREADS; i++)
```

```

printf("Initial counter = %d\n", counter);
/* Create threads */
for (i = 0; i < NUM_THREADS; i++)
{
    if (pthread_create(
        &threads[i],
        NULL,
        increment_counter,
        NULL) != 0)
    {
        perror("pthread_create");
        return 1;
    }
}
/* Wait for all threads */
for (i = 0; i < NUM_THREADS; i++)
{
    pthread_join(threads[i], NULL);
}
printf("Expected counter = %d\n",
    NUM_THREADS * INCREMENTS);
printf("Actual counter   = %d\n", counter);
/* Destroy mutex */
pthread_mutex_destroy(&mutex);
return 0;
}

```

Output:

```

root@Tejasri:~# nano mutex_counter.c
root@Tejasri:~# gcc mutex_counter.c -o mutex_counter
root@Tejasri:~# ./mutex_counter
Initial counter = 0
Expected counter = 500000
Actual counter   = 500000
root@Tejasri:~# |

```